MINERALS REPO - CREATIVE EXPANSION

Generated by: Kimi (Weird Mode Activated)

Date: 2026-05-05

Status: Original minerals + new species, hybrids, and meta-effects

---

PART 1: NEW MINERAL SPECIES (Never Covered)

==========

37. FLUORITE (The Octahedral Rainbow Trap)

==========

Domain: Cubic Cleavage, Zonal Color Banding **Category:** Calcium Fluoride

Technique: Cubic Zoning + UV Fluorescence

Fluorite grows in perfect cubes that cleave into octahedrons. The color zones are growth rings—like tree rings but chemical. Under UV it fluoresces blue-violet. We render the cleavage planes as internal mirrors.

```
precision highp float;
uniform float u_time;
uniform vec2 u_resolution;
uniform vec2 u_mouse;

// Cubic cleavage: 4 planes at tetrahedral angles
float cleavage(vec3 p) {
    vec3 a = abs(p);
    // The 4 cleavage planes of fluorite
    float p1 = abs(a.x + a.y + a.z - 1.0);
    float p2 = abs(a.x + a.y - a.z - 1.0);
    float p3 = abs(a.x - a.y + a.z - 1.0);
    float p4 = abs(-a.x + a.y + a.z - 1.0);
    return min(min(p1, p2), min(p3, p4));
}

// Zonal color: concentric cubic shells
vec3 fluorite_zone(vec3 p) {
    float zone = floor(length(p) * 3.0);
    // Famous fluorite colors: purple, green, blue, yellow
    vec3 colors[4];
    colors[0] = vec3(0.4, 0.0, 0.6);  // Deep purple
    colors[1] = vec3(0.0, 0.5, 0.2);  // Emerald green
    colors[2] = vec3(0.1, 0.3, 0.8);  // Ocean blue
```

```
    colors[3] = vec3(0.8, 0.7, 0.0);  // Golden yellow
    return colors[int(mod(zone, 4.0))];
}

void main() {
    vec2 uv = (gl_FragCoord.xy * 2.0 - u_resolution.xy) / u_resolution.y;
    vec3 p = vec3(uv, 0.0);

    // Octahedral rotation
    float a = u_time * 0.1;
    p.xy *= rot(a);
    p.xz *= rot(a * 0.7);

    // Cleavage planes
    float cleave = cleavage(p);
    float cleave_mask = smoothstep(0.05, 0.0, cleave);

    // Zonal color
    vec3 zone_color = fluorite_zone(p);

    // Internal reflection on cleavage
    float internal_bounce = pow(cleave_mask, 4.0) * 2.0;

    // UV fluorescence toggle (mouse = UV torch)
    float uv_dist = length(uv - (u_mouse / u_resolution - 0.5) * 2.0);
    float uv_hit = smoothstep(0.5, 0.0, uv_dist);
    vec3 fluorescence = vec3(0.3, 0.1, 0.8) * uv_hit * 3.0; // Violet glow

    vec3 final = zone_color * (1.0 + internal_bounce) + fluorescence;
    final += vec3(0.5, 0.5, 1.0) * cleave_mask * 0.5; // Glassy sheen

    gl_FragColor = vec4(final, 1.0);
}
```

Key: Perfect cubes that want to be octahedrons. Chemical growth rings = color zones. UV fluorescence = hidden violet world.

==========

38. AZURITE-MALACHITE (The Copper Gradient)

==========

Domain: Pseudomorphosis, Chemical Gradient **Category:** Copper Carbonate Hydroxide

Technique: Reaction-Diffusion Mineral Front

Azurite (blue) and malachite (green) are the same chemistry at different pH. Azurite is unstable—over geological time it "creeps" into malachite. We render this as a moving chemical front.

```
precision highp float;
```

```glsl
uniform float u_time;
uniform vec2 u_resolution;

// Reaction-diffusion approximation for mineral front
float chemical_front(vec2 p, float t) {
    // Azurite (blue) retreats, malachite (green) advances
    float wave = sin(p.x * 2.0 + p.y * 1.5 + t * 0.3);
    float noise = fbm(p * 3.0 + t * 0.1);
    return smoothstep(-0.2, 0.2, wave + noise * 0.5);
}

void main() {
    vec2 uv = (gl_FragCoord.xy * 2.0 - u_resolution.xy) / u_resolution.y;

    // Banding from both minerals
    float band_azurite = sin(uv.x * 8.0 + fbm(uv * 2.0) * 2.0);
    float band_malachite = sin(uv.y * 6.0 + fbm(uv * 1.5 + 100.0) * 3.0);

    // Chemical front position
    float front = chemical_front(uv, u_time);

    // Colors
    vec3 azurite = vec3(0.05, 0.15, 0.6);  // Deep azure
    vec3 malachite = vec3(0.05, 0.4, 0.15); // Rich green
    vec3 transition = vec3(0.1, 0.3, 0.4);  // Teal boundary

    // Mix based on front
    vec3 color = mix(azurite, malachite, front);
    color = mix(color, transition, smoothstep(0.4, 0.6, front) * smoothstep(0.6, 0.4, front));

    // Concentric banding from each
    float bands = smoothstep(-0.2, 0.2, band_azurite) * (1.0 - front)
                + smoothstep(-0.2, 0.2, band_malachite) * front;
    color *= 0.7 + bands * 0.6;

    // Fibrous malachite texture on green side
    float fibers = pow(abs(sin(uv.x * 50.0 + uv.y * 30.0)), 4.0);
    color += malachite * fibers * front * 0.3;

    gl_FragColor = vec4(color, 1.0);
}
```

Key: Same chemistry, different pH. Blue retreats, green advances. Moving chemical front over millions of years.

==========

39. SHATTUCKITE (The Fibrous Blue Vein)

==========

Domain: Acicular Radiation, Copper Silicate **Category:** Inosilicate

Technique: Radial Fiber Burst with Chatoyancy

Shattuckite forms radiating sprays of deep blue fibers. More intense blue than azurite, with a silky luster from the fiber structure.

```
precision highp float;
uniform float u_time;
uniform vec2 u_resolution;

void main() {
    vec2 uv = (gl_FragCoord.xy * 2.0 - u_resolution.xy) / u_resolution.y;
    float r = length(uv);
    float theta = atan(uv.y, uv.x);

    // Multiple radiation centers
    vec2 centers[3];
    centers[0] = vec2(0.0, 0.0);
    centers[1] = vec2(0.4, -0.3);
    centers[2] = vec2(-0.3, 0.4);

    vec3 final = vec3(0.02, 0.05, 0.1); // Dark matrix

    for(int i = 0; i < 3; i++) {
        vec2 local = uv - centers[i];
        float lr = length(local);
        float ltheta = atan(local.y, local.x);

        // Radiating fibers
        float fibers = 0.0;
        for(int f = 1; f < 6; f++) {
            float freq = float(f) * 15.0;
            fibers += sin(ltheta * freq + u_time * 0.2) * (1.0 / float(f));
        }

        // Fiber mask
        float mask = smoothstep(0.5, 0.0, lr) * smoothstep(-0.5, 0.5, fibers);

        // Shattuckite blue: intense, slightly violet
        vec3 blue = vec3(0.0, 0.2, 0.5) * mask;
        vec3 silk = vec3(0.3, 0.4, 0.7) * pow(mask, 4.0) * 2.0; // Silky highlight

        final += blue + silk;
    }

    // Matrix texture
    float matrix_noise = fbm(uv * 4.0);
    final += vec3(0.1, 0.08, 0.05) * matrix_noise * 0.2;

    gl_FragColor = vec4(final, 1.0);
```

```
}
```

Key: Intense copper blue. Radiating sprays. Silky luster from fiber structure.

==========

40. CHRYSOBERYL (Cat's Eye / Cymophane)

==========

Domain: Chatoyancy, Needle Inclusions **Category:** Beryllium Aluminum Oxide

Technique: Single Sharp Chatoyant Band

Unlike Tiger's Eye's broad bands, chrysoberyl cat's eye has ONE razor-sharp band that moves like a spotlight across the surface. Caused by parallel needle inclusions.

```
precision highp float;
uniform float u_time;
uniform vec2 u_resolution;
uniform vec2 u_mouse;

void main() {
    vec2 uv = (gl_FragCoord.xy * 2.0 - u_resolution.xy) / u_resolution.y;

    // Honey-gold base
    vec3 base = vec3(0.6, 0.5, 0.1);

    // Needle direction (slightly wavy)
    float needle_angle = 0.3 + sin(uv.y * 2.0) * 0.05;
    vec2 needle_dir = vec2(cos(needle_angle), sin(needle_angle));

    // The cat's eye band: sharp, single, moving
    float band_pos = dot(uv, needle_dir);
    float light_pos = dot((u_mouse / u_resolution - 0.5) * 2.0, needle_dir);

    // Band follows light position
    float band = exp(-pow((band_pos - light_pos) * 20.0, 2.0));

    // Sharp highlight (much sharper than Tiger's Eye)
    vec3 highlight = vec3(1.0, 0.9, 0.6) * band * 3.0;

    // Milk-and-honey body color
    float body = 0.8 + sin(uv.x * 3.0 + uv.y * 2.0) * 0.1;

    vec3 final = base * body + highlight;

    // Translucency at edges
    float edge = 1.0 - length(uv) * 0.3;
    final *= edge;
```

```
    gl_FragColor = vec4(final, 1.0);
}
```

Key: ONE sharp band, not many. Moves like spotlight. Milk-and-honey body.

==========

41. ULEXITE (TV Rock / Fiber Optic Mineral)

==========

Domain: Fiber Optics, Image Transmission **Category:** Borate Mineral

Technique: Parallel Fiber Image Transmission

Ulexite transmits images through its internal fiber structure. Place text on paper behind it, see it on top surface. We simulate this by projecting "through" the mineral.

```
precision highp float;
uniform float u_time;
uniform vec2 u_resolution;
uniform sampler2D u_back_image; // Image behind the mineral

void main() {
    vec2 uv = (gl_FragCoord.xy * 2.0 - u_resolution.xy) / u_resolution.y;

    // Ulexite projects image from bottom to top surface
    // Simulate by sampling "back" image with slight distortion
    vec2 back_uv = uv * 0.8 + 0.1; // Slightly cropped

    // Fiber optic distortion: slight magnification and blur
    float fiber_noise = fbm(uv * 20.0) * 0.02;
    back_uv += fiber_noise;

    // Sample the "transmitted" image
    vec3 transmitted = texture2D(u_back_image, back_uv).rgb;

    // Ulexite has silky white fibers
    float fibers = pow(abs(sin(uv.x * 100.0)), 0.1) * 0.3;
    vec3 fiber_color = vec3(0.9, 0.9, 0.85) * fibers;

    // Combine: transmitted image + fiber texture
    vec3 final = transmitted * 0.8 + fiber_color;

    // Slight translucency
    float alpha = 0.9;

    gl_FragColor = vec4(final, alpha);
}
```

Key: Natural fiber optics. Projects image from one surface to another.

==========

42. SELENITE (Desert Rose / Gypsum Flower)

==========

Domain: Evaporite Crystal, Desert Formation **Category:** Calcium Sulfate Dihydrate

Technique: Evaporative Growth Simulation

Selenite forms in desert playas as water evaporates. The crystals grow in flat, blade-like shapes that cluster into "roses." We simulate evaporative crystal growth.

```
precision highp float;
uniform float u_time;
uniform vec2 u_resolution;

// Desert rose petal SDF
float sdPetal(vec2 p, float angle, float length, float width) {
    vec2 local = p;
    local *= rot(-angle);
    // Flattened ellipse
    float d = length(local * vec2(1.0 / width, 1.0 / length)) - 1.0;
    return d;
}

void main() {
    vec2 uv = (gl_FragCoord.xy * 2.0 - u_resolution.xy) / u_resolution.y;

    // Cluster of petals radiating from center
    float cluster = 1e10;
    float num_petals = 8.0;

    for(float i = 0.0; i < num_petals; i++) {
        float angle = i / num_petals * 6.28318 + u_time * 0.05;
        float length = 0.6 + sin(i * 2.0) * 0.1;
        float width = 0.15 + cos(i * 3.0) * 0.05;

        float petal = sdPetal(uv, angle, length, width);
        cluster = smin(cluster, petal, 0.05);
    }

    // Gypsum is translucent white with silky luster
    float inside = smoothstep(0.02, -0.02, cluster);
    float edge = smoothstep(0.05, 0.0, abs(cluster));

    vec3 gypsum = vec3(0.95, 0.93, 0.9);
    vec3 shadow = vec3(0.7, 0.68, 0.65);
    vec3 highlight = vec3(1.0, 0.98, 0.95);

    vec3 final = mix(shadow, gypsum, inside);
```

```
    final += highlight * edge * 2.0; // Silky edge sheen

    // Sand matrix
    float sand = fbm(uv * 8.0 + 50.0);
    vec3 sand_color = vec3(0.8, 0.7, 0.5) * sand;
    final = mix(sand_color, final, inside + edge);

    gl_FragColor = vec4(final, 1.0);
}
```

Key: Desert rose = evaporative growth. Flat blades in radial cluster. Silky gypsum luster.

==========

43. TOURMALINE (The Piezoelectric Rainbow)

==========

Domain: Pyroelectricity, Pleochroism **Category:** Cyclosilicate

Technique: Color-Shift by Crystal Axis

Tourmaline shows different colors along different crystal axes. Also piezoelectric—generates charge under pressure. We render the axis-dependent color shift.

```
precision highp float;
uniform float u_time;
uniform vec2 u_resolution;
uniform vec2 u_mouse;

void main() {
    vec2 uv = (gl_FragCoord.xy * 2.0 - u_resolution.xy) / u_resolution.y;

    // Tourmaline crystal: long prismatic with triangular cross-section
    float r = length(uv);
    float theta = atan(uv.y, uv.x);

    // Triangular cross-section (3-fold symmetry)
    float tri = abs(sin(theta * 3.0)) * r;
    float crystal = smoothstep(0.7, 0.68, tri);

    // C-axis color (lengthwise): usually dark
    vec3 c_axis = vec3(0.1, 0.05, 0.08);

    // A-axis color (crosswise): the famous watermelon tourmaline
    float axis_angle = theta + u_time * 0.2;
    vec3 a_axis = vec3(
        0.5 + 0.5 * sin(axis_angle),
        0.2 + 0.3 * cos(axis_angle * 2.0),
        0.1
    );
```

```
    // Mix based on viewing angle relative to crystal axis
    float view_angle = dot(normalize(uv), vec2(1.0, 0.0));
    vec3 color = mix(c_axis, a_axis, abs(view_angle));

    // Watermelon effect: green outside, pink inside
    float center = smoothstep(0.3, 0.0, r);
    vec3 green_rind = vec3(0.1, 0.4, 0.1);
    vec3 pink_core = vec3(0.6, 0.2, 0.3);
    color = mix(color, mix(green_rind, pink_core, center), crystal);

    // Piezoelectric flash on click
    float pressure = smoothstep(0.2, 0.0, length(uv - (u_mouse / u_resolution - 0.5) * 2.0));
    color += vec3(0.5, 0.5, 1.0) * pressure * 2.0;

    gl_FragColor = vec4(color * crystal, 1.0);
}
```

Key: Axis-dependent color. Watermelon = green rind, pink core. Piezoelectric flash.

==========

44. OBSIDIAN (Volcanic Glass / Conchoidal Fracture)

==========

Domain: Amorphous Solid, Fracture Mechanics **Category:** Volcanic Glass

Technique: Conchoidal Fracture + Flow Banding

Obsidian is glass, not crystal. It breaks with conchoidal (shell-like) fractures. Has flow banding from viscous lava movement.

```
precision highp float;
uniform float u_time;
uniform vec2 u_resolution;

// Conchoidal fracture pattern
float conchoidal(vec2 p, vec2 center, float scale) {
    vec2 local = (p - center) * scale;
    float r = length(local);
    float theta = atan(local.y, local.x);
    // Shell-like curves
    float shell = r - sin(theta * 3.0 + r * 5.0) * 0.1;
    return smoothstep(0.5, 0.0, shell);
}

void main() {
    vec2 uv = (gl_FragCoord.xy * 2.0 - u_resolution.xy) / u_resolution.y;

    // Flow banding from viscous lava
```

```
    float flow = sin(uv.x * 3.0 + uv.y * 5.0 + fbm(uv * 2.0) * 2.0);
    float bands = smoothstep(-0.3, 0.3, flow);

    // Multiple fracture centers
    float fracture = 0.0;
    for(int i = 0; i < 5; i++) {
        vec2 center = vec2(
            sin(float(i) * 2.5 + u_time * 0.1) * 0.5,
            cos(float(i) * 1.7) * 0.5
        );
        fracture += conchoidal(uv, center, 3.0 + float(i));
    }

    // Obsidian colors: black, mahogany, snowflake (cristobalite inclusions)
    vec3 black = vec3(0.02, 0.02, 0.03);
    vec3 mahogany = vec3(0.15, 0.05, 0.03);
    vec3 snowflake = vec3(0.9, 0.9, 0.85);

    // Flow bands = color variation
    vec3 color = mix(black, mahogany, bands);

    // Fracture edges = sharp highlight
    float edge = smoothstep(0.1, 0.0, fracture);
    color += vec3(0.3, 0.3, 0.4) * edge * 2.0; // Glassy edge

    // Snowflake inclusions (random white spots)
    float snow = step(0.97, fbm(uv * 15.0));
    color = mix(color, snowflake, snow * 0.3);

    // Glassy reflection
    float gloss = pow(1.0 - abs(uv.x), 8.0) * 0.3;
    color += vec3(0.2, 0.2, 0.3) * gloss;

    gl_FragColor = vec4(color, 1.0);
}
```

Key: Volcanic glass. Conchoidal = shell-like fractures. Flow bands from viscous lava.

==========

45. MOLDAVITE (Tektite / Impact Glass)

==========

Domain: Impact Metamorphism, Aerodynamic Forming **Category:** Natural Glass

Technique: Aerodynamic Sculpting + Bubble Inclusions

Moldavite is glass formed by meteorite impact. It has distinctive aerodynamic shapes from flying through the atmosphere while molten. Full of bubbles and lechatelierite (silica glass) inclusions.

```
precision highp float;
uniform float u_time;
uniform vec2 u_resolution;

// Aerodynamic teardrop SDF
float sdTeardrop(vec2 p, float scale) {
    p *= scale;
    float r = length(p);
    float theta = atan(p.y, p.x);
    // Teardrop: round front, pointed back
    float shape = r - (1.0 - cos(theta) * 0.3) * smoothstep(3.14, 0.0, theta);
    return shape;
}

void main() {
    vec2 uv = (gl_FragCoord.xy * 2.0 - u_resolution.xy) / u_resolution.y;

    // Rotate to show aerodynamic form
    float a = u_time * 0.1;
    vec2 p = uv * rot(a);

    // Main teardrop body
    float body = sdTeardrop(p, 1.5);
    float inside = smoothstep(0.02, -0.02, body);
    float edge = smoothstep(0.05, 0.0, abs(body));

    // Moldavite green: olive to forest
    vec3 green = vec3(0.2, 0.4, 0.15);
    vec3 dark_green = vec3(0.1, 0.25, 0.08);

    // Surface texture: wrinkled from flight
    float wrinkles = fbm(p * 5.0 + vec2(100.0, 50.0)) * 0.3;
    vec3 color = mix(dark_green, green, 0.5 + wrinkles);

    // Bubble inclusions (lechatelierite)
    float bubbles = 0.0;
    for(int i = 0; i < 8; i++) {
        vec2 b_pos = vec2(
            sin(float(i) * 3.1 + 1.0) * 0.4,
            cos(float(i) * 2.7 + 2.0) * 0.3
        );
        float b = smoothstep(0.08, 0.0, length(p - b_pos));
        bubbles += b;
    }

    // Bubbles = lighter, with internal reflection
    vec3 bubble_color = vec3(0.5, 0.6, 0.4) * bubbles;
    color += bubble_color * inside;

    // Edge highlight (molten glass sheen)
```

```
    color += vec3(0.3, 0.4, 0.2) * edge * 2.0;

    // Translucency
    float alpha = 0.85 + edge * 0.15;

    gl_FragColor = vec4(color * inside, alpha);
}
```

Key: Meteorite impact glass. Aerodynamic from flying molten. Bubble inclusions.

PART 2: HYBRID TECHNIQUES (Crossbreeding Existing Methods)

==========

H1. BRAGG-SCATTERING + KIFS FOLDING (Labradorite-Amethyst Hybrid)

==========

Combine the interference flash of labradorite with the recursive void-space of amethyst. Result: a geode that flashes spectral colors from its internal crystal faces.

```
// In amethyst KIFS loop, add:
vec3 crystal_normal = normalize(pc);
float interference = dot(viewDir, crystal_normal);
float flash = pow(abs(interference), 20.0);
vec3 spectral = pal(interference * 2.0, ...);
// Add to violet glow: violet + spectral * flash
```

==========

H2. ANISOTROPIC + THIN-FILM (Tiger's Eye + Turgite)

==========

Tiger's eye fiber structure with turgite's curvature-driven interference. Result: fibers that change color along their length based on local curvature.

```
// In Tiger's Eye tangent loop:
float fiber_curvature = sin(uv.x * 5.0) * 0.5 + 0.5;
float phase = dot(view, tangent) * 2.0 + fiber_curvature * 3.0;
vec3 fiber_color = spectral_palette(phase);
// Replace tiger_palette with this
```

==========

H3. VORONOI + ANISOTROPIC (Opal + Stibnite)

==========

Opal's domain structure with stibnite's needle geometry. Result: opal with acicular (needle-like) domains instead of spherical.

```
// In opal Voronoi loop, replace sphere distance with needle distance:
float needle_d = sdNeedle(f - point, vec3(0.0, 0.0, 1.0), 0.3, 0.02);
if(needle_d < min_dist) {
    domain_normal = normalize(point - f);
    min_dist = needle_d;
}
```

==========

H4. FEEDBACK + BRAGG (Vivianite + Spectrolite)

==========

Vivianite's oxidation memory with spectrolite's full-spectrum interference. Result: a mineral that "remembers" light exposure by shifting its interference pattern permanently.

```
// Use exposure buffer to shift spectrolite's phase:
float phase_shift = exposure * 3.14; // Permanent phase offset
float schiller = pow(max(0.0, alignment + phase_shift), 64.0);
```

==========

H5. POLAR + L1 NORM (Cummingtonite + Bismuth)

==========

Cummingtonite's radial sunburst with bismuth's Manhattan geometry. Result: radiating square-stepped needles.

```
// In cummingtonite radial loop:
float manhattan_r = abs(local.x) + abs(local.y);
float square_needle = step(0.5, sin(theta * freq)) * step(manhattan_r, 0.5);
```

PART 3: MINERAL-ADJACENT PHENOMENA (Not Rocks, But Rock-Like)

==========

P1. FROST CRYSTALS (Windowpane Ice)

==========

Domain: Crystallization, Dendritic Growth **Category:** Meteorological Mineral

Dendritic ice crystals on glass. Different from manganese dendrites—these are true fractals with 6-fold symmetry.

```
precision highp float;
uniform float u_time;
```

```
uniform vec2 u_resolution;
uniform float u_temperature; // Colder = more complex

// 6-fold dendritic branch
float ice_branch(vec2 p, float angle, float scale, float depth) {
    if(depth <= 0.0) return 0.0;

    vec2 local = p * scale;
    local *= rot(-angle);

    // Main stem
    float stem = smoothstep(0.05, 0.0, abs(local.y)) * step(0.0, local.x) * step(local.x, 1.0);

    // Side branches at 60 degrees
    float branch1 = ice_branch(local - vec2(0.5, 0.0), 1.047, scale * 0.6, depth - 1.0);
    float branch2 = ice_branch(local - vec2(0.5, 0.0), -1.047, scale * 0.6, depth - 1.0);

    return stem + branch1 * 0.7 + branch2 * 0.7;
}

void main() {
    vec2 uv = (gl_FragCoord.xy * 2.0 - u_resolution.xy) / u_resolution.y;

    // Multiple seed points
    float frost = 0.0;
    for(int i = 0; i < 6; i++) {
        vec2 seed = vec2(
            sin(float(i) * 2.3) * 0.7,
            cos(float(i) * 1.9) * 0.7
        );
        float branch = ice_branch(uv - seed, atan(uv.y - seed.y, uv.x - seed.x), 2.0, 4.0);
        frost += branch;
    }

    // Ice colors: clear to milky white
    vec3 clear_ice = vec3(0.85, 0.9, 0.95);
    vec3 frosty = vec3(0.95, 0.97, 1.0);

    vec3 final = mix(clear_ice, frosty, frost);
    final += vec3(1.0, 1.0, 1.1) * frost * 0.5; // Bright edges

    // Glass background
    vec3 glass = vec3(0.7, 0.8, 0.9) * (1.0 - length(uv) * 0.2);
    final = mix(glass, final, smoothstep(0.0, 0.5, frost));

    gl_FragColor = vec4(final, 1.0);
}
```

Key: True 6-fold dendritic growth. Recursive branching. Temperature controls complexity.

==========

P2. SALT FLATS (Halite Polygonal Cracks)

==========

Domain: Evaporite, Desiccation Cracking **Category:** Sedimentary Surface

Salt flats form polygonal patterns as water evaporates and salt crystallizes. The cracks form from thermal expansion/contraction.

```
precision highp float;
uniform float u_time;
uniform vec2 u_resolution;

// Voronoi with hexagonal bias (salt polygons)
float salt_polygons(vec2 p) {
    vec2 i = floor(p);
    vec2 f = fract(p);
    float d = 1.0;

    for(int y = -1; y <= 1; y++) {
        for(int x = -1; x <= 1; x++) {
            vec2 b = vec2(float(x), float(y));
            // Hexagonal offset
            b.x += mod(i.y + b.y, 2.0) * 0.5;
            vec2 r = b + hash21(i + b) - f;
            d = min(d, dot(r, r));
        }
    }
    return sqrt(d);
}

void main() {
    vec2 uv = (gl_FragCoord.xy * 2.0 - u_resolution.xy) / u_resolution.y;

    float scale = 4.0;
    float poly = salt_polygons(uv * scale);

    // Polygon edges = raised salt ridges
    float ridge = smoothstep(0.1, 0.0, poly);
    float flat = smoothstep(0.3, 0.1, poly);

    // Salt colors: white crust, pink from algae, wet dark patches
    vec3 salt_white = vec3(0.95, 0.95, 0.92);
    vec3 salt_pink = vec3(0.9, 0.75, 0.7);
    vec3 wet = vec3(0.4, 0.4, 0.5);

    // Wet patches in polygon centers
    float wetness = hash21(floor(uv * scale));
    vec3 color = mix(salt_white, salt_pink, wetness);
    color = mix(color, wet, smoothstep(0.7, 0.9, wetness) * flat);
```

```
    // Raised ridges
    color += vec3(0.1, 0.1, 0.08) * ridge;

    // Specular from salt crystals
    float spec = pow(max(0.0, 1.0 - poly * 3.0), 32.0) * flat;
    color += vec3(1.0) * spec * 0.3;

    gl_FragColor = vec4(color, 1.0);
}
```

Key: Hexagonal Voronoi = salt polygons. Raised ridges at edges. Wet/dry variation.

==========

P3. LIMESTONE CAVERN (Stalactite/Stalagmite Dripping)

==========

Domain: Speleothem, Chemical Precipitation **Category:** Cave Formation

Calcite dripping and building formations over millennia. We simulate the drip-by-drip growth.

```
precision highp float;
uniform float u_time;
uniform vec2 u_resolution;

// Stalactite SDF: cone with drip
float sdStalactite(vec2 p, vec2 tip, float length, float width) {
    vec2 local = p - tip;
    local.y = -local.y; // Point down
    float taper = smoothstep(0.0, length, local.y);
    float w = width * (1.0 - taper * 0.7);
    float body = length(vec2(local.x, 0.0)) - w * (1.0 - local.y / length);

    // Drip at tip
    float drip = length(p - (tip + vec2(sin(u_time) * 0.05, length))) - 0.03;

    return min(body, drip);
}

void main() {
    vec2 uv = (gl_FragCoord.xy * 2.0 - u_resolution.xy) / u_resolution.y;

    // Ceiling with multiple stalactites
    float cave = 1e10;
    for(int i = 0; i < 5; i++) {
        vec2 tip = vec2(
            sin(float(i) * 2.5) * 0.6,
            0.8 + cos(float(i) * 1.3) * 0.1
        );
```

```
        float stal = sdStalactite(uv, tip, 0.6 + sin(float(i)) * 0.2, 0.08);
        cave = min(cave, stal);
    }

    // Floor stalagmites (upside down)
    for(int i = 0; i < 4; i++) {
        vec2 tip = vec2(
            cos(float(i) * 2.1) * 0.5,
            -0.8 + sin(float(i) * 1.7) * 0.1
        );
        float stal = sdStalactite(vec2(uv.x, -uv.y), tip, 0.5, 0.1);
        cave = min(cave, stal);
    }

    float inside = smoothstep(0.02, -0.02, cave);
    float edge = smoothstep(0.05, 0.0, abs(cave));

    // Calcite colors: cream to orange (from iron)
    vec3 calcite = vec3(0.9, 0.85, 0.75);
    vec3 iron_stain = vec3(0.6, 0.4, 0.2);
    float stain = fbm(uv * 3.0) * 0.5;
    vec3 color = mix(calcite, iron_stain, stain);

    // Wet sheen
    color += vec3(0.2, 0.2, 0.15) * edge * 2.0;

    // Cave darkness
    float depth = 1.0 - abs(uv.y) * 0.5;
    color *= depth;

    gl_FragColor = vec4(color * inside, 1.0);
}
```

Key: Drip-by-drip growth. Stalactite (ceiling) vs stalagmite (floor). Iron staining.

==========

P4. BIOLUMINESCENT MINERAL (Fantasy Hybrid)

==========

Domain: Biomineralization, Chemiluminescence **Category:** Fantasy/Speculative

What if minerals could glow like fireflies? We combine the structure of calcite with the chemistry of bioluminescence.

```
precision highp float;
uniform float u_time;
uniform vec2 u_resolution;
uniform vec2 u_mouse;
```

```
// Bioluminescent "bacteria" colonies on mineral surface
float biolum_colony(vec2 p, vec2 center, float phase) {
    float d = length(p - center);
    // Pulsing glow
    float pulse = sin(u_time * 2.0 + phase) * 0.5 + 0.5;
    return exp(-d * d * 20.0) * pulse;
}

void main() {
    vec2 uv = (gl_FragCoord.xy * 2.0 - u_resolution.xy) / u_resolution.y;

    // Mineral base: rough calcite texture
    float rough = fbm(uv * 4.0);
    vec3 mineral = vec3(0.7, 0.75, 0.8) * (0.8 + rough * 0.4);

    // Bioluminescent colonies
    float glow = 0.0;
    vec3 glow_color = vec3(0.0, 0.0, 0.0);

    for(int i = 0; i < 12; i++) {
        vec2 center = vec2(
            sin(float(i) * 3.7 + 1.0) * 0.7,
            cos(float(i) * 2.3 + 2.0) * 0.6
        );
        float colony = biolum_colony(uv, center, float(i));
        glow += colony;

        // Different colonies = different colors (speciation)
        vec3 colors[3];
        colors[0] = vec3(0.0, 1.0, 0.5); // Cyan-green
        colors[1] = vec3(0.5, 0.0, 1.0); // Violet
        colors[2] = vec3(0.0, 0.5, 1.0); // Blue
        glow_color += colors[i % 3] * colony;
    }

    // Combine: mineral absorbs some glow, transmits rest
    vec3 final = mineral * (1.0 - glow * 0.3) + glow_color * 2.0;

    // Mouse disturbs colonies (scares them = dimmer)
    float mouse_dist = length(uv - (u_mouse / u_resolution - 0.5) * 2.0);
    float scare = smoothstep(0.3, 0.0, mouse_dist);
    final *= 1.0 - scare * 0.5;

    gl_FragColor = vec4(final, 1.0);
}
```

Key: Speculative. Mineral + bioluminescence. Pulsing colonies. Mouse interaction.

PART 4: LIGHTING EFFECTS (What Minerals Do To Light)

==========

L1. CAUSTICS THROUGH CRYSTAL (Water + Quartz)

==========

Light passing through a quartz crystal into water creates dancing caustics. We render the caustic pattern, not the crystal.

```
precision highp float;
uniform float u_time;
uniform vec2 u_resolution;

// Caustic pattern from refracted light
float caustics(vec2 p, float time) {
    float c = 0.0;
    for(int i = 0; i < 3; i++) {
        float fi = float(i);
        vec2 q = p * (2.0 + fi);
        q += time * (0.5 + fi * 0.2);
        c += sin(q.x + sin(q.y + time)) * cos(q.y - sin(q.x));
    }
    return c * 0.5 + 0.5;
}

void main() {
    vec2 uv = (gl_FragCoord.xy * 2.0 - u_resolution.xy) / u_resolution.y;

    // Pool bottom
    vec3 pool = vec3(0.1, 0.3, 0.4);

    // Caustic intensity
    float caustic = caustics(uv * 3.0, u_time * 0.5);
    caustic = pow(caustic, 3.0); // Sharpen

    // Color shift from refraction (prism effect)
    vec3 caustic_color = vec3(
        caustics(uv * 3.0 + vec2(0.02, 0.0), u_time * 0.5),
        caustic,
        caustics(uv * 3.0 - vec2(0.02, 0.0), u_time * 0.5)
    );
    caustic_color = pow(caustic_color, vec3(3.0));

    // Combine
    vec3 final = pool + caustic_color * 2.0;

    // Water surface reflection
    float surface = pow(1.0 - abs(uv.y), 8.0) * 0.3;
    final += vec3(0.5, 0.7, 0.8) * surface;
```

```
    gl_FragColor = vec4(final, 1.0);
}
```

Key: Render the effect, not the cause. Dancing light on pool bottom.

==========

L2. DISPERSION PRISM (White Light → Spectrum)

==========

Minerals with high dispersion (diamond, sphalerite) split white light into rainbows. We render the split.

```
precision highp float;
uniform float u_time;
uniform vec2 u_resolution;
uniform vec2 u_mouse;

void main() {
    vec2 uv = (gl_FragCoord.xy * 2.0 - u_resolution.xy) / u_resolution.y;

    // White light source position
    vec2 light_pos = (u_mouse / u_resolution - 0.5) * 2.0;

    // Distance from light
    float d = length(uv - light_pos);

    // Dispersion: different wavelengths refract differently
    // Red bends least, violet bends most
    float dispersion_strength = 0.3 / (d + 0.1);

    vec3 color = vec3(0.0);
    for(int i = 0; i < 7; i++) {
        float wavelength = float(i) / 6.0; // 0=red, 1=violet
        float bend = (wavelength - 0.5) * dispersion_strength;

        vec2 refracted = normalize(uv - light_pos) * bend;
        float intensity = exp(-length(uv - light_pos - refracted) * 10.0);

        // Spectrum colors
        vec3 spectrum = vec3(
            smoothstep(0.5, 0.0, abs(wavelength - 0.0)), // Red
            smoothstep(0.5, 0.0, abs(wavelength - 0.3)), // Green
            smoothstep(0.5, 0.0, abs(wavelength - 0.6))  // Blue
        );
        color += spectrum * intensity;
    }

    // Background
    vec3 bg = vec3(0.05, 0.05, 0.08);
    float beam = exp(-d * d * 2.0) * 0.5;
```

```
    gl_FragColor = vec4(bg + color + vec3(1.0) * beam, 1.0);
}
```

Key: White light splitting into spectrum. Different wavelengths = different refraction.

==========

L3. POLARIZED LIGHT THROUGH CALCITE (Double Refraction)

==========

Calcite is birefringent—it splits light into two paths with different polarizations. We simulate the double image.

```
precision highp float;
uniform float u_time;
uniform vec2 u_resolution;
uniform sampler2D u_scene; // Background scene

void main() {
    vec2 uv = gl_FragCoord.xy / u_resolution.xy;

    // Ordinary ray (no shift)
    vec3 ordinary = texture2D(u_scene, uv).rgb;

    // Extraordinary ray (shifted based on crystal orientation)
    float crystal_angle = u_time * 0.1;
    vec2 shift = vec2(cos(crystal_angle), sin(crystal_angle)) * 0.02;
    vec3 extraordinary = texture2D(u_scene, uv + shift).rgb;

    // Interference between the two (polarization effect)
    float phase = sin(u_time) * 0.5 + 0.5;
    vec3 combined = mix(ordinary, extraordinary, phase);

    // Rainbow fringes at edges (chromatic aberration from birefringence)
    float edge = length(shift) * 50.0;
    vec3 fringe = vec3(
        smoothstep(0.0, 0.3, edge),
        smoothstep(0.1, 0.4, edge),
        smoothstep(0.2, 0.5, edge)
    );

    gl_FragColor = vec4(combined + fringe * 0.2, 1.0);
}
```

Key: Double image from birefringence. Polarization interference. Rainbow fringes.

==========

L4. X-RAY DIFFRACTION PATTERN (Reciprocal Space)

==========

What a mineral looks like to X-rays, not visible light. Laue pattern or powder diffraction.

```
precision highp float;
uniform float u_time;
uniform vec2 u_resolution;

// Bragg peaks for cubic lattice
float bragg_peak(vec2 p, vec3 hkl, float wavelength) {
    // d-spacing for cubic: a / sqrt(h²+k²+l²)
    float d = 1.0 / sqrt(hkl.x*hkl.x + hkl.y*hkl.y + hkl.z*hkl.z);
    float theta = asin(wavelength / (2.0 * d));

    // Peak position on detector
    float peak_pos = tan(theta * 2.0) * 2.0;
    float dist = abs(length(p) - peak_pos);
    return exp(-dist * dist * 100.0);
}

void main() {
    vec2 uv = (gl_FragCoord.xy * 2.0 - u_resolution.xy) / u_resolution.y;

    float pattern = 0.0;
    vec3 color = vec3(0.0);

    // Miller indices for common peaks
    vec3 hkls[6];
    hkls[0] = vec3(1,0,0); hkls[1] = vec3(1,1,0);
    hkls[2] = vec3(1,1,1); hkls[3] = vec3(2,0,0);
    hkls[4] = vec3(2,1,0); hkls[5] = vec3(2,1,1);

    float wavelength = 0.15; // Cu K-alpha in nm-ish units

    for(int i = 0; i < 6; i++) {
        float peak = bragg_peak(uv, hkls[i], wavelength);
        pattern += peak;

        // Color by intensity (structure factor)
        float intensity = 1.0;
        if(int(hkls[i].x + hkls[i].y + hkls[i].z) % 2 == 1) intensity = 0.0; // Systematic absence

        color += vec3(0.5 + float(i) * 0.1, 0.8, 1.0) * peak * intensity;
    }

    // Direct beam (center)
    float direct = exp(-length(uv) * length(uv) * 50.0);

    // Film background
    vec3 film = vec3(0.1, 0.15, 0.2);
```

```
    vec3 final = film + color * 2.0 + vec3(1.0) * direct * 0.5;

    gl_FragColor = vec4(final, 1.0);
}
```

Key: X-ray's eye view. Bragg peaks. Miller indices. Systematic absences.

PART 5: TEXTURE ATLAS (Reusable Mineral Surface Patterns)

==========

T1. CONCHOIDAL FRACTURE PATTERN

==========

Reusable for obsidian, flint, chert, glass.

```
float conchoidal_fracture(vec2 p, vec2 center, float scale, float roughness) {
    vec2 local = (p - center) * scale;
    float r = length(local);
    float theta = atan(local.y, local.x);

    // Shell-like curves with roughness
    float shell = r - sin(theta * 3.0 + r * 5.0 + fbm(local * 3.0) * roughness) * 0.1;
    return smoothstep(0.5, 0.0, shell);
}
```

==========

T2. FIBROUS/RADIATING PATTERN

==========

Reusable for shattuckite, okenite, cummingtonite, stibnite.

```
float radiating_fibers(vec2 p, vec2 center, float num_fibers, float freq, float time) {
    vec2 local = p - center;
    float r = length(local);
    float theta = atan(local.y, local.x);

    float fibers = 0.0;
    for(float i = 0.0; i < num_fibers; i++) {
        float f = (i / num_fibers) * 6.28318;
        float fiber = sin(theta * freq + f + time) * exp(-r * 2.0);
        fibers += max(0.0, fiber);
    }
    return fibers;
}
```

==========

T3. BANDED/LAYERED PATTERN

==========

Reusable for agate, fordite, malachite, banded iron.

```
float banded_layers(vec2 p, float direction, float frequency, float warp_strength) {
    // Domain warp for organic banding
    vec2 q = p;
    q += vec2(
        sin(q.y * 2.0 + p.x) * warp_strength,
        cos(q.x * 1.5 + p.y) * warp_strength
    );

    // Project to band direction
    float proj = q.x * cos(direction) + q.y * sin(direction);
    return sin(proj * frequency);
}
```

==========

T4. CRYSTAL FACE/PRISM PATTERN

==========

Reusable for quartz, fluorite, calcite, tourmaline.

```
float crystal_faces(vec2 p, float symmetry, float size) {
    float r = length(p);
    float theta = atan(p.y, p.x);

    // Polygonal cross-section
    float polygon = cos(theta * symmetry) * size;
    float faces = smoothstep(polygon * 0.9, polygon, r);

    // Facet boundaries
    float facets = abs(sin(theta * symmetry));
    float edges = smoothstep(0.1, 0.0, facets);

    return faces + edges * 0.3;
}
```

==========

T5. BOTRYOIDAL/GRAPE PATTERN

==========

Reusable for hematite, goethite, uraninite, chalcedony.

```
float botryoidal_cluster(vec2 p, float num_spheres, float radius, float merge) {
    float d = 1e10;
```

```
    for(float i = 0.0; i < num_spheres; i++) {
        vec2 center = vec2(
            sin(i * 2.5 + 1.0) * 0.5,
            cos(i * 1.7 + 2.0) * 0.5
        );
        float sphere = length(p - center) - radius * (0.8 + sin(i) * 0.2);
        d = smin(d, sphere, merge);
    }
    return d;
}
```

PART 6: ADVANCED/META CONCEPTS

==========

M1. MINERAL TIME-LAPSE (Growth Animation)

==========

A shader that grows a crystal from seed to full form over time.

```
// Seed → Nucleation → Growth → Coarsening → Equilibrium
float growth_stage(float time) {
    if(time < 0.2) return 0.0; // Seed
    if(time < 0.4) return smoothstep(0.2, 0.4, time); // Nucleation
    if(time < 0.7) return 1.0; // Growth
    if(time < 0.9) return 1.0 - smoothstep(0.7, 0.9, time) * 0.3; // Coarsening
    return 0.7; // Equilibrium
}

// Use growth_stage to control crystal size, facet sharpness, inclusion density
```

==========

M2. MINERAL WEATHERING (Surface Degradation)

==========

Fresh crystal → etched → dissolved → replaced.

```
float weathering(float time, float hardness) {
    // Hardness 0-1: 0=dissolves fast, 1=resists
    float etch = smoothstep(0.0, 0.3 * hardness, time);
    float dissolve = smoothstep(0.3 * hardness, 1.0, time);
    float replacement = smoothstep(0.8, 1.0, time);
    return etch + dissolve * 2.0 + replacement * 3.0;
}

// Apply as surface roughness, edge rounding, color shift
```

==========

M3. MULTI-MINERAL PARAGENESIS (Mineral Sequence)

==========

Multiple minerals growing in sequence, replacing each other.

```
// Paragenetic sequence: quartz → sulfides → carbonates → oxides
vec3 paragenesis(vec2 p, float depth) {
    float stage = fract(depth * 0.1);

    if(stage < 0.25) return quartz_color(p);      // Early: quartz vein
    if(stage < 0.5)  return sulfide_color(p);     // Middle: pyrite, galena
    if(stage < 0.75) return carbonate_color(p);   // Late: calcite, dolomite
    return oxide_color(p);                         // Supergene: goethite
}
```

==========

M4. MINERAL OPTICAL MICROSCOPE (Thin Section)

==========

What geologists see: crossed polarizers, interference colors, mineral identification.

```
// Thin section under crossed polars
vec3 thin_section(vec2 p, float birefringence, float thickness, float orientation) {
    // Interference color from retardation
    float retardation = birefringence * thickness;
    float phase = retardation * 6.28318;

    // Michel-Levy color chart approximation
    vec3 color = vec3(
        sin(phase) * 0.5 + 0.5,
        sin(phase + 2.09) * 0.5 + 0.5,
        sin(phase + 4.19) * 0.5 + 0.5
    );

    // Extinction when oriented
    float extinction = pow(abs(sin(orientation * 2.0)), 4.0);

    return color * extinction;
}
```

SUMMARY

New Minerals Added: 9

- Fluorite, Azurite-Malachite, Shattuckite, Chrysoberyl, Ulexite, Selenite, Tourmaline, Obsidian, Moldavite

Hybrid Techniques: 5

- Bragg+KIFS, Anisotropic+ThinFilm, Voronoi+Anisotropic, Feedback+Bragg, Polar+L1Norm

Mineral-Adjacent Phenomena: 4

- Frost crystals, Salt flats, Limestone caverns, Bioluminescent mineral

Lighting Effects: 4

- Caustics, Dispersion prism, Polarized light, X-ray diffraction

Texture Atlas: 5 reusable patterns

- Conchoidal, Fibrous, Banded, Crystal faces, Botryoidal

Meta Concepts: 4

- Growth animation, Weathering, Paragenesis, Thin section

Total: 31 new items beyond the original 36